Prédiction des Prix de Maison

Machine Learning appliqué à la régression

Guide pratique pas à pas

Nettoyage des données & Évaluation du modèle

Ce que vous allez construire

Un pipeline complet de régression : de l'exploration des données brutes jusqu'à un modèle évalué rigoureusement, prêt à être amélioré.

Niveau : débutant → intermédiaire

Outils : Python, pandas, scikit-learn

Table des matières

1 Vue d'ensemble du projet **2**
1.1 Jeux de données recommandés . . . 2
1.2 Le pipeline global . . . 2
1.3 Objectifs pédagogiques . . . 2

2 Exploration des données (EDA) **3**
2.1 Premier contact avec les données . . . 3
2.2 Analyse de la variable cible . . . 3
2.3 Relations entre variables . . . 4

3 Nettoyage des données **5**
3.1 Valeurs manquantes . . . 5
3.1.1 Identifier les valeurs manquantes . . . 5
3.1.2 Stratégies d'imputation . . . 5
3.2 Valeurs aberrantes (outliers) . . . 6
3.2.1 Détection . . . 6
3.2.2 Traitement . . . 6
3.3 Encodage des variables catégorielles . . . 7
3.4 Feature engineering . . . 7
3.5 Colinéarité entre variables . . . 7
3.6 Construire un pipeline de nettoyage reproductible . . . 8

4 Préparation pour la modélisation **9**
4.1 Séparation train / test . . . 9
4.2 Mise à l'échelle des variables . . . 9

5 Modélisation **10**
5.1 Modèle de référence (baseline) . . . 10
5.2 Modèles régularisés . . . 10
5.3 Modèles à base d'arbres . . . 10

6 Évaluation du modèle **12**
6.1 Métriques de régression . . . 12
6.2 Validation croisée . . . 12
6.3 Analyse des résidus . . . 13
6.4 Courbes d'apprentissage . . . 13
6.5 Importance des variables . . . 14
6.6 Pièges courants d'évaluation . . . 14

7 Aller plus loin **15**
7.1 Réglage des hyperparamètres . . . 15
7.2 Combiner plusieurs modèles . . . 15
7.3 Sauvegarder le modèle . . . 15

Fiche récapitulative **16**

1 Vue d'ensemble du projet

L'objectif de ce projet est de construire un modèle capable de prédire le prix de vente d'une maison à partir de ses caractéristiques (surface, nombre de pièces, localisation, année de construction, etc.). C'est l'un des exercices les plus classiques en machine learning supervisé, car il combine des données réalistes, plusieurs types de variables (numériques et catégorielles), des valeurs manquantes, et une vraie question métier : *à combien estimer un bien ?*

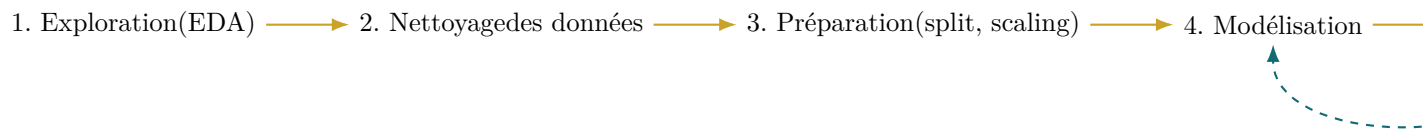
1.1 Jeux de données recommandés

Trois jeux de données sont couramment utilisés pour ce type de projet, du plus simple au plus riche :

- **California Housing** (intégré à scikit-learn via `fetch_california_housing`) : peu de nettoyage nécessaire, bon point de départ.
- **Ames Housing** (Kaggle, *House Prices - Advanced Regression Techniques*) : très riche (79 variables), beaucoup de valeurs manquantes et de variables catégorielles — idéal pour pratiquer le nettoyage en profondeur.
- **Boston Housing** : historique mais déprécié (biais éthiques documentés) — à éviter pour un nouveau projet.

Ce guide s'appuie sur la structure d'un dataset de type **Ames Housing**, car c'est celui qui illustre le mieux les problématiques réelles de nettoyage de données.

1.2 Le pipeline global



Note

La flèche en pointillés rappelle que l'évaluation n'est jamais la dernière étape : elle sert à décider s'il faut revenir en arrière (nettoyer davantage, changer de modèle, créer de nouvelles variables).

1.3 Objectifs pédagogiques

- Savoir diagnostiquer et traiter des données réelles imparfaites.
- Construire un pipeline reproductible (pas de nettoyage "à la main" non traçable).
- Choisir et interpréter les bonnes métriques d'évaluation pour un problème de régression.
- Éviter les pièges classiques : fuite de données, overfitting, métriques trompeuses.

2 Exploration des données (EDA)

Avant de nettoyer quoi que ce soit, il faut comprendre ce que l'on a entre les mains. Cette étape guide toutes les décisions de nettoyage qui suivent.

2.1 Premier contact avec les données

```
import pandas as pd
import numpy as np

df = pd.read_csv("train.csv")

print(df.shape)           # (nb_lignes, nb_colonnes)
print(df.info())          # types, valeurs non-nulles
print(df.describe())      # statistiques des variables numeriques
print(df.head())
```

Points à vérifier systématiquement :

- Le nombre de lignes et de colonnes est-il cohérent avec ce qui est attendu ?
- Quels types de données pandas a-t-il inféré (`int64`, `float64`, `object`) ? Un `object` peut cacher une variable catégorielle ou une erreur de saisie.
- Y a-t-il des colonnes qui ressemblent à un identifiant (`Id`) et qu'il faudra exclure du modèle ?

2.2 Analyse de la variable cible

Le prix (`SalePrice`) mérite une attention particulière car c'est ce qu'on essaie de prédire.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df["SalePrice"], kde=True)
plt.title("Distribution du prix de vente")
plt.show()

print("Skewness :", df["SalePrice"].skew())
```

Définition — Asymétrie (skewness)

La *skewness* mesure l'asymétrie d'une distribution. Les prix immobiliers sont presque toujours **asymétriques à droite** (quelques biens très chers tirent la distribution vers les grandes valeurs). Une skewness positive élevée (> 1) suggère qu'une transformation logarithmique de la cible ($\log(1 + \text{prix})$) rapprochera la distribution d'une gaussienne, ce qui aide beaucoup de modèles linéaires.

```
df["SalePrice_log"] = np.log1p(df["SalePrice"])
```

2.3 Relations entre variables

```
# Correlation des variables numeriques avec la cible
corr = df.corr(numeric_only=True)["SalePrice"].sort_values(ascending=False)
print(corr.head(10))

# Matrice de correlation visuelle (variables les plus corrélées)
top_features = corr.index[1:11]
sns.heatmap(df[top_features].corr(), annot=True, cmap="viridis")
```

Note

La corrélation linéaire ne détecte que les relations *linéaires*. Une variable comme la qualité du quartier (catégorielle) peut être très prédictive sans jamais apparaître dans une matrice de corrélation numérique — d'où l'intérêt de croiser avec des boxplots (prix par catégorie).

3 Nettoyage des données

C'est le cœur du travail sur un jeu de données réel : un modèle, même sophistiqué, ne peut pas compenser des données mal préparées (*garbage in, garbage out*).

3.1 Valeurs manquantes

3.1.1 Identifier les valeurs manquantes

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).sort_values(ascending=False)
print(missing_pct[missing_pct > 0])
```

Piège à éviter

Sur des datasets immobiliers type Ames Housing, une valeur manquante dans une colonne comme `PoolQC` (qualité de la piscine) ne signifie **pas** "donnée perdue" — elle signifie **"pas de piscine"**. Supprimer ces lignes ou imputer une moyenne serait une erreur de logique métier, pas seulement statistique. Il faut toujours lire le dictionnaire de données avant de choisir une stratégie.

3.1.2 Stratégies d'imputation

Stratégie	Quand l'utiliser	Risque
Suppression de la colonne	Plus de 40–50% de valeurs manquantes et faible valeur prédictive	Perte d'information
Suppression des lignes	Très peu de lignes concernées, données MCAR	Perte de données si mal ciblé
Imputation moyenne/médiane	Variable numérique, distribution simple	Réduit la variance réelle
Imputation par mode	Variable catégorielle	Idem, écrase la diversité
Catégorie "Absent"	NaN = absence réelle (ex : pas de garage)	Aucun si logique confirmée
KNNImputer / itératif	Variables corrélées entre elles	Plus coûteux, fuite si mal découpé

```
from sklearn.impute import SimpleImputer

# Cas 1 : NaN = absence reelle (variable categorielle)
cols_absence = ["PoolQC", "Alley", "Fence", "GarageType"]
df[cols_absence] = df[cols_absence].fillna("Aucun")

# Cas 2 : imputation numerique par la mediane (robuste aux outliers)
num_imputer = SimpleImputer(strategy="median")
df[["LotFrontage"]] = num_imputer.fit_transform(df[["LotFrontage"]])

# Cas 3 : imputation categorielle par le mode
cat_imputer = SimpleImputer(strategy="most_frequent")
df[["Electrical"]] = cat_imputer.fit_transform(df[["Electrical"]])
```

Piège à éviter — fuite de données

Un `SimpleImputer` (ou tout autre transformateur) doit être **ajusté (`fit`) uniquement sur les données d'entraînement**, puis appliqué (`transform`) sur le jeu de test. Si vous imputez avant de séparer train/test avec des statistiques calculées sur l'ensemble complet, vous laissez fuiter de l'information du test vers l'entraînement. La bonne pratique, détaillée en section 4, est d'utiliser un `Pipeline` scikit-learn.

3.2 Valeurs aberrantes (outliers)

3.2.1 Détection

```
# Methode IQR (interquartile range)
Q1 = df["GrLivArea"].quantile(0.25)
Q3 = df["GrLivArea"].quantile(0.75)
IQR = Q3 - Q1
borne_basse = Q1 - 1.5 * IQR
borne_haute = Q3 + 1.5 * IQR

outliers = df[(df["GrLivArea"] < borne_basse) | (df["GrLivArea"] > borne_haute)]
print(f"{len(outliers)} outliers detectes sur GrLivArea")

# Visualisation
sns.scatterplot(x="GrLivArea", y="SalePrice", data=df)
```

Définition — Règle de l'IQR

$IQR = Q3 - Q1$ (écart entre le 3e et le 1er quartile). Une observation est considérée comme aberrante si elle est en dehors de l'intervalle $[Q1 - 1.5 \times IQR,\ Q3 + 1.5 \times IQR]$. C'est une convention statistique, pas une loi absolue : elle doit être confrontée au sens métier des données.

3.2.2 Traitement

- **Suppression** : justifiée si l'observation est manifestement une erreur de saisie (ex : maison de 50 m² vendue 5 millions).
- **Capping / winsorizing** : on remplace les valeurs extrêmes par les bornes de l'IQR plutôt que de perdre la ligne entière.
- **Transformation logarithmique** : réduit l'impact des grandes valeurs sans supprimer d'information (souvent la meilleure option pour les surfaces et les prix).

Exemple — cas réel Ames Housing

Sur ce dataset, deux maisons ont une surface habitable (`GrLivArea`) énorme mais un prix de vente anormalement bas. Ce ne sont pas des erreurs de mesure mais des ventes atypiques (ventes forcées, biens partiels). La documentation officielle du dataset recommande explicitement de les retirer de l'entraînement.

3.3 Encodage des variables catégorielles

Type d'encodage	Principe	Cas d'usage
One-Hot Encoding	Une colonne binaire par catégorie	Catégories sans ordre (ex : type de toit)
Ordinal Encoding	Un entier par catégorie, ordre respecté	Catégories ordonnées (ex : qualité Ex/Gd/TA/Fa/Po)
Target Encoding	Moyenne de la cible par catégorie	Variables à forte cardinalité, avec précaution

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# Variable ordonnee : qualite generale
qualite_ordre = [["Po", "Fa", "TA", "Gd", "Ex"]]
ord_enc = OrdinalEncoder(categories=qualite_ordre)
df["ExterQual_enc"] = ord_enc.fit_transform(df[["ExterQual"]])

# Variable non ordonnee : type de quartier
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
neighborhood_encoded = ohe.fit_transform(df[["Neighborhood"]])
```

Piège à éviter — Target Encoding

Le *target encoding* utilise la variable cible pour créer la feature : si on l'applique sur tout le dataset avant le split, on introduit une fuite massive (la feature "connaît" déjà la réponse). Il faut soit l'appliquer uniquement après le split avec un `fit` sur le train, soit utiliser une version avec validation croisée intégrée (ex : `category_encoders.TargetEncoder`).

3.4 Feature engineering

Créer de nouvelles variables informatives à partir des colonnes existantes améliore souvent plus la performance qu'un changement d'algorithme.

```
# Age du bien au moment de la vente
df["AgeMaison"] = df["YrSold"] - df["YearBuilt"]

# Surface totale (sous-sol + rez-de-chaussee + etage)
df["SurfaceTotale"] = df["TotalBsmtSF"] + df["1stFlrSF"] + df["2ndFlrSF"]

# A ete renove ou non
df["ARenove"] = (df["YearRemodAdd"] != df["YearBuilt"]).astype(int)

# Ratio salle de bain / chambres
df["RatioSdb"] = df["FullBath"] / df["BedroomAbvGr"].replace(0, 1)
```

3.5 Colinéarité entre variables

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X_num = df[["GrLivArea", "TotalBsmtSF", "SurfaceTotale", "GarageArea"]].dropna()
vif = pd.DataFrame()
```

```
vif["variable"] = X_num.columns
vif["VIF"] = [variance_inflation_factor(X_num.values, i) for i in range(X_num.shape
    [1])]
print(vif)
```

Définition — VIF (Variance Inflation Factor)

Le VIF mesure à quel point une variable explicative est corrélée aux autres variables explicatives. Un VIF supérieur à 5 (parfois 10 selon la convention retenue) indique une forte colinéarité : les coefficients d'une régression linéaire deviennent instables. Solution : supprimer une des variables redondantes, ou utiliser un modèle régularisé (Ridge/Lasso) qui tolère mieux la colinéarité.

3.6 Construire un pipeline de nettoyage reproductible

Note

Plutôt que d'enchaîner des transformations "à la main" sur le `DataFrame`, il est fortement recommandé d'encapsuler le nettoyage dans un `ColumnTransformer`. Cela garantit que les mêmes transformations, ajustées uniquement sur le train, sont appliquées de façon identique au test et à toute donnée future.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

colonnes_num = ["GrLivArea", "SurfaceTotale", "AgeMaison", "GarageArea"]
colonnes_cat = ["Neighborhood", "ExterQual", "GarageType"]

pipeline_num = Pipeline([
    ("imputation", SimpleImputer(strategy="median")),
    ("normalisation", StandardScaler())
])

pipeline_cat = Pipeline([
    ("imputation", SimpleImputer(strategy="constant", fill_value="Aucun")),
    ("encodage", OneHotEncoder(handle_unknown="ignore"))
])

preprocesseur = ColumnTransformer([
    ("num", pipeline_num, colonnes_num),
    ("cat", pipeline_cat, colonnes_cat)
])
```

4 Préparation pour la modélisation

4.1 Séparation train / test

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["SalePrice", "SalePrice_log", "Id"])
y = df["SalePrice_log"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Piège à éviter

Tout ce qui a été montré en section 3 (imputation, encodage, création de variables statistiques) doit désormais être `fit` sur `X_train` uniquement. Le `Pipeline` scikit-learn du bloc précédent résout ce problème automatiquement : appelez `pipeline.fit(X_train, y_train)` puis `pipeline.transform(X_test)`, jamais l'inverse.

4.2 Mise à l'échelle des variables

Scaler	Principe	Quand l'utiliser
StandardScaler	Centre-réduit (moyenne 0, écart-type 1)	Modèles linéaires, distances (KNN, SVM)
MinMaxScaler	Ramène entre 0 et 1	Réseaux de neurones, bornes connues
RobustScaler	Utilise médiane/IQR	Données avec outliers non supprimés

Note

Les modèles à base d'arbres (Random Forest, Gradient Boosting) sont **insensibles à l'échelle des variables** : la mise à l'échelle est inutile pour eux, mais indispensable pour la régression linéaire, Ridge, Lasso, KNN ou SVM.

5 Modélisation

5.1 Modèle de référence (baseline)

Toujours commencer par un modèle simple : il donne un point de comparaison et révèle si les modèles complexes apportent un gain réel.

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

modele_baseline = Pipeline([
    ("preprocesseur", preprocesseur),
    ("regresseur", LinearRegression())
])

modele_baseline.fit(X_train, y_train)
```

5.2 Modèles régularisés

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

modeles = {
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.001),
    "ElasticNet": ElasticNet(alpha=0.001, l1_ratio=0.5)
}
```

Définition — Pourquoi régulariser ?

Ridge (pénalité L_2) et Lasso (pénalité L_1) ajoutent une contrainte sur la taille des coefficients pour limiter le surapprentissage, particulièrement utile quand il y a beaucoup de variables (après le One-Hot Encoding, on peut facilement dépasser 100 colonnes). Le Lasso a la particularité de pouvoir amener certains coefficients exactement à zéro : il fait implicitement de la sélection de variables.

5.3 Modèles à base d'arbres

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

rf = RandomForestRegressor(n_estimators=300, max_depth=12, random_state=42)
gb = GradientBoostingRegressor(n_estimators=300, learning_rate=0.05, random_state
    =42)
```

Modèle	**Avantages**	**Limites**
Régression linéaire	Simple, interprétable, rapide	Suppose des relations linéaires
Ridge / Lasso	Robuste à la colinéarité, peu de variance	Toujours linéaire
Random Forest	Capture les non-linéarités, peu de réglages	Moins interprétable, plus lourd
Gradient Boosting	Souvent la meilleure performance brute	Sensible au surapprentissage, plus de réglages

6 Évaluation du modèle

Un modèle qui n'est pas évalué correctement peut sembler excellent alors qu'il ne fonctionnera pas en conditions réelles. Cette section est le pendant indispensable du nettoyage : les deux déterminent si les résultats obtenus sont dignes de confiance.

6.1 Métriques de régression

Métrique	Formule	Interprétation
MAE	$\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i \rvert$	Erreur moyenne absolue, même unité que la cible, peu sensible aux outliers
MSE	$\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$	Pénalise fortement les grosses erreurs
RMSE	$\sqrt{\text{MSE}}$	Comme MSE mais dans l'unité d'origine, la plus utilisée en pratique
R^2	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$	Proportion de variance expliquée (1 = parfait, 0 = aussi bon qu'une moyenne)
MAPE	$\frac{1}{n}\sum_i \left\lvert \frac{y_i - \hat{y}_i}{y_i} \right\rvert$	Erreur relative en %, utile pour comparer entre gammes de prix

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = modele_baseline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)

print(f"MAE  : {mae:.4f}")
print(f"RMSE : {rmse:.4f}")
print(f"R2   : {r2:.4f}")
```

Note — quelle métrique choisir ?

Si le prix cible a été transformé en $\log(1+\text{prix})$, pensez à repasser en échelle réelle (`np.expm1`) avant de calculer MAE/RMSE si vous voulez un chiffre interprétable en dollars ou en dirhams. Pour comparer objectivement des modèles entre eux, le RMSE et le R^2 suffisent ; pour communiquer à un public non technique, le MAPE (en %) est souvent plus parlant.

6.2 Validation croisée

Un seul split train/test dépend du hasard du découpage. La validation croisée moyenne les résultats sur plusieurs découpages pour une estimation plus fiable.

```
from sklearn.model_selection import cross_val_score, KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
scores = cross_val_score(
    modele_baseline, X_train, y_train,
    scoring="neg_root_mean_squared_error", cv=kf
)

print("RMSE moyen :", -scores.mean())
print("Ecart-type :", scores.std())
```

Définition — K-Fold

Le K-Fold découpe les données d'entraînement en K parties (*folds*). Le modèle est entraîné K fois, chaque fois sur $K-1$ parties et évalué sur la partie restante. On obtient ainsi K scores, dont la moyenne est une estimation plus robuste de la performance que celle obtenue avec un seul split, et dont l'écart-type renseigne sur la stabilité du modèle.

6.3 Analyse des résidus

```
residus = y_test - y_pred

# Residus vs predictions : doit ressembler a un nuage sans structure
plt.scatter(y_pred, residus, alpha=0.5)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predictions")
plt.ylabel("Residus")
plt.title("Residus vs predictions")

# QQ-plot : verifie la normalite des residus
import scipy.stats as stats
stats.probplot(residus, dist="norm", plot=plt)
```

Note — comment lire ces graphiques

- Un nuage de résidus **sans forme particulière**, centré autour de zéro, est un bon signe (homoscédasticité).
- Un **entonnoir** (résidus qui augmentent avec la prédiction) indique une hétéroscédasticité : le modèle est moins fiable sur les prix élevés — souvent corrigé par la transformation logarithmique de la cible.
- Une **courbe** dans les résidus vs prédictions suggère une relation non-linéaire que le modèle linéaire ne capture pas.

6.4 Courbes d'apprentissage

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, val_scores = learning_curve(
    modele_baseline, X_train, y_train,
    cv=5, scoring="neg_root_mean_squared_error",
    train_sizes=np.linspace(0.1, 1.0, 10)
)

plt.plot(train_sizes, -train_scores.mean(axis=1), label="Train")
plt.plot(train_sizes, -val_scores.mean(axis=1), label="Validation")
plt.xlabel("Taille du jeu d'entrainement")
```

```
plt.ylabel("RMSE")
plt.legend()
```

Signature observée	Diagnostic	Piste de correction
Erreur train et validation élevées, proches	Sous-apprentissage	Modèle plus complexe, plus de features
Erreur train faible, validation élevée, écart important	Surapprentissage	Régularisation, plus de données, simplifier le modèle
Les deux courbes convergent vers une erreur faible	Bon équilibre	Passer au réglage fin (hyperparamètres)

6.5 Importance des variables

```
from sklearn.inspection import permutation_importance

rf.fit(X_train_transforme, y_train)
resultat = permutation_importance(
    rf, X_test_transforme, y_test, n_repeats=10, random_state=42
)

importances = pd.Series(resultat.importances_mean, index=noms_colonnes)
importances.sort_values(ascending=False).head(10).plot(kind="barh")
```

Note

La *permutation importance* mesure de combien la performance du modèle chute quand on mélange aléatoirement une variable — c'est plus fiable que `feature_importances_` natif des arbres, qui peut être biaisé en faveur des variables à haute cardinalité.

6.6 Pièges courants d'évaluation

Piège 1 — fuite de données (data leakage)

Ajuster un `scaler`, un `imputer` ou un `encoder` sur l'ensemble complet des données avant le split fait "fuiter" de l'information du test vers le train. Le score obtenu est alors artificiellement optimiste et ne se reproduira pas en production.

Piège 2 — surapprentissage du jeu de test par le réglage répété

Si vous ajustez vos hyperparamètres en regardant le score sur le jeu de test à chaque essai, vous finissez par "sur-apprendre" ce jeu de test spécifique, même sans jamais l'utiliser pour l'entraînement. La bonne pratique est de réserver un troisième jeu de **validation** (ou d'utiliser la validation croisée) pour le réglage, et de ne toucher au jeu de test qu'une seule fois, à la fin.

Piège 3 — métrique mal choisie

Un R^2 élevé peut masquer une erreur absolue énorme sur les biens les plus chers. Toujours regarder plusieurs métriques ensemble (RMSE *et* MAE *et* l'analyse des résidus), jamais une seule isolément.

7 Aller plus loin

7.1 Réglage des hyperparamètres

```
from sklearn.model_selection import GridSearchCV

grille = {
    "n_estimators": [200, 300, 500],
    "max_depth": [8, 12, 16],
    "min_samples_leaf": [1, 3, 5]
}

recherche = GridSearchCV(
    RandomForestRegressor(random_state=42),
    grille, cv=5, scoring="neg_root_mean_squared_error", n_jobs=-1
)
recherche.fit(X_train_transforme, y_train)

print("Meilleurs parametres :", recherche.best_params_)
```

Note

Pour de grandes grilles de paramètres, `RandomizedSearchCV` explore un échantillon aléatoire de combinaisons plutôt que toutes — souvent presque aussi performant pour une fraction du temps de calcul.

7.2 Combiner plusieurs modèles

```
from sklearn.ensemble import VotingRegressor

modele_final = VotingRegressor([
    ("ridge", Ridge(alpha=1.0)),
    ("rf", RandomForestRegressor(n_estimators=300, random_state=42)),
    ("gb", GradientBoostingRegressor(n_estimators=300, random_state=42))
])
```

7.3 Sauvegarder le modèle

```
import joblib

joblib.dump(modele_final, "modele_prix_maison.joblib")

# Rechargement plus tard
modele_charge = joblib.load("modele_prix_maison.joblib")
```

Fiche récapitulative

Checklist — Nettoyage des données

- ☐ Vérifié le type de chaque colonne (`info()`)
- ☐ Distingué NaN = "absence réelle" vs NaN = "donnée manquante"
- ☐ Choisi une stratégie d'imputation justifiée pour chaque colonne
- ☐ Détecté et traité les outliers (IQR, visualisation)
- ☐ Encodé les variables catégorielles (ordinal si ordre, one-hot sinon)
- ☐ Créé de nouvelles variables pertinentes (feature engineering)
- ☐ Vérifié la colinéarité (VIF, matrice de corrélation)
- ☐ Encapsulé tout le nettoyage dans un `Pipeline`/`ColumnTransformer`

Checklist — Évaluation du modèle

- ☐ Split train/test réalisé **avant** tout ajustement de transformateur
- ☐ Calculé au moins MAE, RMSE et R^2
- ☐ Validation croisée effectuée (pas un seul split)
- ☐ Résidus analysés visuellement (vs prédictions, QQ-plot)
- ☐ Courbes d'apprentissage tracées pour diagnostiquer sur/sous-apprentissage
- ☐ Importance des variables calculée et interprétée
- ☐ Jeu de test utilisé une seule fois, à la toute fin

Formule clé	**Expression**
RMSE	$\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$
R^2	$1 - \frac{\text{SS}_{\text{résidus}}}{\text{SS}_{\text{totale}}}$
IQR (outliers)	$[Q1 - 1.5\,IQR,\ Q3 + 1.5\,IQR]$
Transformation cible	$\log(1 + \text{prix})$, retour via `np.expm1`

Pour résumer

Un bon modèle de prédiction de prix immobilier repose à 70% sur la qualité du nettoyage et de l'ingénierie des variables, et à 30% sur le choix de l'algorithme. Passer du temps sur les sections 3 et 6 de ce guide rapporte généralement plus qu'essayer dix algorithmes différents sur des données mal préparées.